

PJ3 实验文档

一、超参数调优

1、BP

① 学习率

在 pj1 中，我对于 $\sin(x)$ 函数拟合的学习率调整有详细的描述。在做完整个 pj 之后，可以总结出这样的规律：神经网络越复杂时，学习率应该越小；神经网络越简单时，学习率往往会更大。可能的原因是随着网络越发复杂，模型中的参数也變得越多，模型的凹凸变化也变得更加频繁。如果仍然以比较大的幅度调整参数，可能会导致模型错过一些局部最小值，导致学习效果不佳。相对地，较小的学习率使得模型更新更加平缓，更容易找到“最优解”。而且，随着模型深度的增加，更有可能出现梯度消失或梯度爆炸的情况，因为梯度在传播过程中由于链式法则会不断乘以一个较小或较大的数，当乘法次数增多后，这种梯度可能就会超出浮点数的范围。在有了拟合 $\sin(x)$ 函数的经验和上述的经验性规律之后，我在其他实验中寻找合适的学习率的速度就更加快了。

② batch size

随着科技的不断发展，现代的 gpu 已经有了强大的并行计算能力。因此，选择一个合适的 batch size 能够较为充分高效地利用 gpu，提升计算效率。而 gpu 的构建基于二进制，在 batch size 为 2 的次方时效率会比较高，因此在实验中我主要会选择 128 作为 batch size。在 mnist 手写数字识别中，我也尝试了以 112 作为 batch size。尽管在这样一个并不复杂的数据集上，112 的 batch size 并没有体现出很大的劣势，但考虑到 gpu 的底层逻辑，选择 2 的次方数作为 batch size 可能是更好的选择。在 task3 中，我尝试了以 100 作为 batch size，结果表明在相同的参数下，batch size 为 128 时模型的学习效果要更好，具体数据为 batch size 为 100 时正确率为 90.75%，为 128 时正确率为 91.26%。

③ 权重初始化方法

权重初始化往往指的是通过先验知识将参数调整到一个比较合理的状态。然而即使没有这种先验知识，模型也可以通过不断地训练找到较为合理的参数。比如在 task1 中，我并未将 bias 初始化为负数，最终也达到了理想的结果。不过随着网络深度的加深，这种先验知识能够极大地减少模型找到局部最优解的时间，因此合适的权重初始化在深层神经网络中较为重要。

2、CNN

① 对于卷积的理解

卷积相较于全连接层来说具有很多独特的优势，在实验中，我体会到了这些优势：

卷积的参数较少。由于卷积通过卷积核的方式将图片中一小片的区域进行了线性变换，而无论多大的图片，都可以使用同一个卷积核进行卷积，这种方式实现了参数共享，相对于全连接层来说大大减少了计算量。具体来说，在 `pj1` 中使用单纯使用全连接层在 `mnist` 手写数字识别的数据集上训练一轮的时间与 `pj2` 中使用卷积方法训练一轮的时间差不多，并没有因为卷积导致效率上的降低。不仅如此，现在的 `gpu` 性能很强，使用卷积能够较为高效地使用 `gpu` 的并行计算能力。相对地，全连接层则没有这种硬件设备上的优势。因此卷积的计算量比较少，而效率较高。而计算量的减少使得对于一张图片使用不同卷积核成为可能，这也使得人们能够从 `mnist` 的黑白图片识别进一步进入到 `cifar-10` 的 RGB 颜色识别。

卷积非常适合处理 “grid-like data”，比如图片等二维数据。卷积能够更好地学习到局部数据的联系。每一个卷积核只关注一小块二维平面内的数据，这使得模型能够更好地关注到行、列之间的联系。相较于全连接层单纯地将图片展平的向量形式，卷积的二维形式在几何上“天生”地具有优势。不仅如此，卷积可以通过 `padding` 的方式在图片周围进行填充，从而能够进一步学习到图片边角地方的信息。同时，卷积可以控制 `stride` 的大小，让卷积核以一定的步长划过图片。这种方式使得图片即使发生了一定程度上的偏移、翻折、扭曲，卷积仍然能够关注到这些信息，从而极大地提升了模型的鲁棒性。

卷积能够高效地复用数据。通常来说第一层卷积层会关注图片中最为细碎的特征。而通过 `pool` 的方式，在卷积后的 `feature map` 大小以一定倍率缩小了，并将这种缩小后的 `feature map` 输入到下一层卷积中。那么下一层的卷积通过复用信息就可以关注到原有图片中范围更广的信息，也就是更大的“感受野”，从而学习到更为复杂的特征。随着卷积层数的加深，模型学到的特征就更为复杂，能够解决更加复杂的问题。这种复用数据的效率是指数级的，远远超过了全连接层的效率。

除了随着深度增加而学习到不同的特征，对于同一张图片使用不同的卷积核也能够让模型学习到不同的特征。对于较为复杂的图形，人们可以通过增加卷积核数量的方式让不同的卷积核学习不同的信息。比如一个卷积核用于学习图片的纹理，另外一个卷积核学习物体的轮廓，其他的卷积核学习物体的表面细节等等。这样，卷积可以进一步处理更加复杂的问题。

② 对于池化方式的理解

池化是一般会将 `feature map` 的大小降低从而起到保留最重要的特征、减少计算量的作用。最为常用的池化方式是最大池化和平均池化。

一般来说，在卷积层中会采用最大池化。这是因为卷积后更大的数值往往代表了这个区域内最重要的信息。在 `task2` 中的 `mnist` 和 `cifar-10` 中我的池化方式全部选择了最大池化，最后达到了良好的效果。

然而，在使用 `resnet` 的时候，卷积层的最后一层采用了全局平均池化。这是因为经过多层的卷积层，模型已经学习到了较为优秀的表征。此时使用最大池化会丢失特征图的全局信息，而全局平均池化会通过平均的方式在一定程度上保留这些高级特征，从而起到更好的分类效果。

③ 对于 `stride` 的理解

在 `task2` 中的 CNN 中，我让 `stride` 全部等于 1，达到了比较好的效果。在 `stride=1`, `padding=1`, `kernel size=3` 时，`feature map` 的大小能够与输入保持一致。

但是在 `task3` 中的 `resnet` 中，第一个 `make layer` 函数的步长为 1，这是为了保持与输入值相同的尺寸，从而先提取一次初始特征。后续的 `make layer` 函数的步长为 2，这会使得 `feature map` 的尺寸减小，被称作 `downsampling`。这种方式与池化一样能够起到减少计算量和增加后续层的感受野的效果，区别在于步长为 2 的下采样方式在本质上仍然是进行了卷积，因此这种操作的参数是可以学习的。相对的，池化的参数无法学习，只是单纯地比较。

二、 消融实验

1、不同激活函数的影响

在拟合 $\sin(x)$ 函数的时候，我最初选择了 `sigmoid` 函数，但是拟合效果并不好。而在我将 `sigmoid` 替换为 `tanh` 后，学习效果显著增加了。这可能是因为 `sigmoid` 函数会将输入值全部映射到 0, 1 之间，而 `tanh` 会将输入值映射到 -1, 1 之间，从而使得模型能够学习到负相关性，从而增强模型的学习能力。因此对于 $\sin(x)$ 函数这种弯折程度比较大的函数，`tanh` 的学习效果更好。

不仅如此，`tanh` 函数在输入接近 0 时相较于 `sigmoid` 函数的梯度更加陡峭，这使得模型在调整参数时具有更大的梯度，在相同的学习率下能够提升学习效率。

然而，`tanh` 和 `sigmoid` 函数都具有通病，即在输入值远离 0 的时候梯度较小。因此在较为复杂的问题中，如 `cifar-10` 的分类任务中，我选择了 `ReLU` 函数作为激活函数。这种函数的特点是只要输入值为正，函数的梯度就一直为 1，从而使得在更广泛的范围上能够提供较快的梯度更新。不过 `ReLU` 函数将所有负数全部映射为 0，使得一部分的神经元可能一直无法被激活，导致“Dead ReLU”问题，因此人们提出了优化版本 `GeLU` 函数，保留一定的输入值为负数时的特征。

2、对于 `dropout` 的理解

在 `task2` 的较浅层网络中，我在 `cifar-10` 任务中尝试使用了 `dropout` 函数，但是并没有显著的效果提升。这是由于较浅层的网络本身学习能力有限，若是使用 `dropout` 随机关闭一些神经元，可能会导致模型不足以学习到充足的表征。因此在浅层网络中使用 `dropout` 并不一定能提升学习效果，甚至可能起到相反的作用。

然而，在 `task3` 中的 `resnet` 部分，我在 `cifar-10` 中使用了 `dropout`，并且得到了一定

的准确率提升。这说明随着网络深度的增加，**dropout** 确实能够在一定程度上提升学习效果。

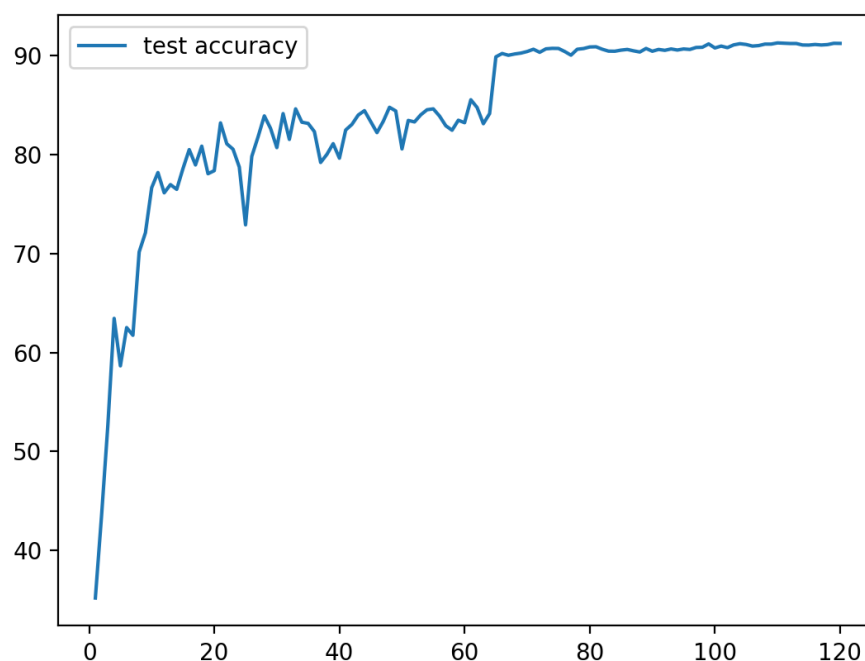
从本质上来说，**dropout** 是一种正则化方法。如同 **weight decay** 通过乘法数值更大的参数来防止过拟合一样，**dropout** 通过简化训练时的神经网络来达到降低复杂程度的效果，从而防止模型学习到噪声和不必要的细节，也降低了梯度消失和爆炸的风险。

3、对于优化器的选择

除了最后的 **resnet** 处理 **cifar-10** 中我选择了 **sgd** 作为优化器，其他情况下我都选择了 **adam** 作为优化器。这是因为 **adam** 能够自适应地调整学习率，同时也结合了动量的思想，使得模型有机会跳出当前的局部最优解，有机会寻找到下一个更优的局部最优解。同时，**adam** 需要设置的参数比较少，对于不复杂的模型来说，直接使用 **adam** 作为优化器已经足够。

在 **resnet** 处理 **cifar-10** 中，我复现了论文中的梯度下降方法，在 **sgd** 中显式添加了 **weight decay** 参数来惩罚较大的参数，使用动量让模型有机会跳出当前的局部最优解。通过这种设置，**sgd** 也具有了 **adam** 的特点，并且更加细致地调整了参数。

同时，我还使用了 **MultiStepLR** 来逐步降低学习率。这种方式使得在训练轮数增大后学习率能够降低，从而使得模型能够更细致地更新参数。从实验上来看，这种更新学习率的方式对应上了准确率随 **epoch** 的变化曲线，在第二次更新学习率后，准确率逐渐趋于平稳。我将这种学习率变化放在了下图。



三、 对于 resnet 的理解

对于神经网络层数显著超过问题复杂程度的情况，按照道理来说它们的表现应该至少不比更浅层的网络差。但是实验表明深层网络的效果会显著低于浅层网络。这种猜测来源于当模型已经达到较好的效果后，剩下的层数只需要作恒等映射就可以保持这种效果到最后。但是实验表明模型很难通过非线性变换得到恒等映射。从数学的角度上来看，随机写出一个矩阵，它是可逆矩阵的概率非常小，因此通过学习的方式学到多个矩阵相乘等于单位矩阵是一种很难的方式。而在 resnet 中，作者将一层中模型需要学习的对象由 $F(x)$ 转换为了 $F(x)-x$ ，并在这一层的输出中再把这个 x 加回来。这样一来，模型只需要将系数全部置为 0 就可以达到线性映射的目的，这种方式对于模型来说更加简单。这种方式能够实现和 highway network 一样的关门效果，并且正确率上更高。而模型通过将参数置 0 的方式能够更加方便地学习到恒等映射，相较于之前的 VGGnet 等更加古老的结构，resnet 能够用更浅的层数就达到更好的效果，从而极大地减少了模型的总参数。这使得训练更深层的神经网络更加方便快捷。

residual 意为残差。在阅读论文的过程中，作者提到了“残差的本质”这一概念，强调了残差是分类问题中顺其自然的方式。如同描述地图上一个点的位置时，寻找一个参照物相比单纯地说明这个位置在距离地图中心多元的距离来说要更加直观。所以残差是机器学习中非常自然又非常重要的方式。

在 50 层以上的 resnet 中，作者使用了 bottleneck 结构，将原来的两层 3×3 卷积改成 $1 \times 1 + 3 \times 3 + 1 \times 1$ 的方式。 1×1 的卷积能够降低 feature map 的通道数，并在结束时将通道数再改回去，从而能够进一步减少 3×3 卷积中需要的计算量和参数量，进一步方便深层神经网络的训练。

在我的实验中，我将 resnet 应用在了 mnist 上，正确率很快就达到了 99% 以上。不过 mnist 数据集相对简单，即使不使用 resnet 正确率仍然相当高，因此 resnet 的表现并没有特别明显。而在 cifar-10 任务中，我使用了 $2 + 2 \times 3 \times 2 = 14$ 层的 resnet，将正确率从原本不使用 resnet 时的 82% 提升到了 90%，可以看出 resnet 在更难的任务中表现更加明显。由于个人设备运算能力的限制，我只进一步尝试了 $2 + 2 \times 3 \times 3 = 20$ 层的 resnet，最终正确率超过了 91%，效果很好。